Next (and you will need to have the `graphviz dot` [https://graphviz.org/docs/layouts/dot/] command installed), you can generate an image to, for example, render the goroutine call graphs, using

```
$ go tool pprof -png http://localhost:6060/debug/pprof/goroutine > gr.png

Fetching profile over HTTP from http://localhost:6060/debug/pprof/goroutine
Saved profile in ~/tmp/pprof/pprof.goroutine.002.pb.gz
```

OK, that web view or the image may or may not be insightful, but there is another (more popular) way to use `pprof`: the interactive mode (think shell). Let's do that (gathering CPU profiles for 30 seconds, so you will need to wait a little bit):

```
$ go tool pprof "http://localhost:6060/debug/pprof/profile?seconds=30"

Fetching profile over HTTP from
http://localhost:6060/debug/pprof/profile?seconds=30
Saved profile in ~/pprof/pprof.samples.cpu.002.pb.gz
Type: cpu
Time: Mar 8, 2023 at 07:01am (GMT)
Duration: 30.13s, Total samples = 29.55s (98.09%)
Entering interactive mode (type "help" for commands, "o" for options)
...
```

At this point, you're in the `pprof` interactive shell, and you can list the top 50 functions (called *nodes* here) and use the cumulative focus with the `top 50 cum` command like so (edited to show the most important bits):

```
(pprof) top 50 cum
Active filters:
   focus=cum
Showing nodes accounting for 15.54s, 52.59% of 29.55s total
Dropped 47 nodes (cum <= 0.15s)
      flat  flat%   sum%        cum   cum%
     3.72s 12.59% 12.59%      3.73s 12.62%  fmt.(*fmt).fmtSbx
     2.12s  7.17% 19.76%      4.15s 14.04%  runtime.mallocgc
     1.44s  4.87% 24.64%      1.44s  4.87%  crypto/sha256.sha256block
       ...
     0.12s  0.41% 50.19%     14.59s 49.37%  main.slowTask
       ...
         0     0% 52.59%      1.73s  5.85%  main.quickTask
```

From the output of the `top` command in `pprof`, you can tell that the `slowTask()` function (over the 30-second sampling period) took up almost 50% of the runtime and the `quickTask()` function in the same time took up less than 6%. I won't go into further detail here, since the purpose of this exercise is not to teach you Go's `pprof` tooling but to give you an idea of the effort involved.

So let us recap what we had to do to see something at all: